Authenticated chat agents

Himali Tea House and Lotus Wellness Studio

Technical notes | 2 November 2026

1. Purpose

These notes describe the running application: how a person logs in, how the two shops stay apart, how a chat turn is handled, how notes are retrieved, and which HTTP routes exist. They follow the code in *shop/* and the pages in *web/*.

2. The two shops

One FastAPI process and one Postgres database serve both shops. A row always belongs to one organisation. The shop is chosen by the URL. The organisation is chosen by the session, not by a field in the request body.

Shop	Page	URL slug	Organisation
Himali Tea House	/tea	himali-tea	org_a
Lotus Wellness Studio	/studio	lotus-studio	org_b

The home page at / lists both shops. Opening */tea* does not sign the browser into the studio. The client keeps one token per shop, under the keys *shop-token:tea* and *shop-token:studio*.

Where	Address
Local pages	http://127.0.0.1:5173
Local API	http://127.0.0.1:8000
Live site	https://multi-tenant-chat-agents.vercel.app

3. Login

The client sends *POST /login* with an email and a password. The server looks up the email in the tea shop, then in the studio. A match returns a bearer token, the organisation id, and the role. The password is stored as a bcrypt hash. *POST /logout* deletes that session row.

Signup is *POST /shops/{slug}/signup*. The shop comes from the URL. Signup always creates a customer. It never creates an admin.

Later calls send the token in the header *Authorization: Bearer <token>*.

Development accounts. The password for each one is devpass.

Email	Shop	Role
bishal@tea.com	Himali Tea House	Customer
admin@tea.com	Himali Tea House	Admin
bishal@studio.com	Lotus Wellness Studio	Customer
admin@studio.com	Lotus Wellness Studio	Admin

4. Isolation

The shop is read from the URL slug. The organisation is read from the session row. A shop id in the JSON body, or in a header, does not change the shop. If the session belongs to the other shop, the API returns 403 and the message "This login is for another shop."

Caller	Request	Result
Tea customer	Studio customer chat	403
Tea token	GET /me/bookings	403
Admin	Customer chat	403
Customer	Admin chat, or an admin table	403

Orders, bookings, conversations, and the knowledge search all use *WHERE org_id* equal to the session organisation. If a model passes *orgId*, *org_id*, *customerId*, or *now*, those fields are dropped. The session values are the ones that are used.

5. How a customer chat runs

Customer chat is *POST /shops/{slug}/chat/stream*. The body is JSON, with *message* and, on a later turn, *conversationId*. The response is a stream of server-sent events: *meta*, *token*, *tool*, *citation*, *done*, and *error*. The meta event carries the conversation id. If the client does not send one, the server creates an id that starts with *chat_*.

The agent is a LangGraph ReAct agent. The chat model is Groq. The model name comes from *GROQ_CHAT_MODEL*. Short-term memory is the last 12 turns of that conversation.

A write, such as an order or a booking, is not saved on the first turn. The tool returns *confirm_required*. The server holds that pending write until the user sends a plain yes. Only then is the row inserted. Any other reply leaves the row unsaved.

Tea tools cover products, orders, and the shop notes. Studio tools cover services, time slots, bookings, and the shop notes. Each tool calls the shop rules, then Postgres.

Figure 1. One customer chat turn

Read from Start at the upper left. A failed check stops at 401 or 403. A read tool returns to the agent. A write waits for a plain yes, and only that yes inserts a row.

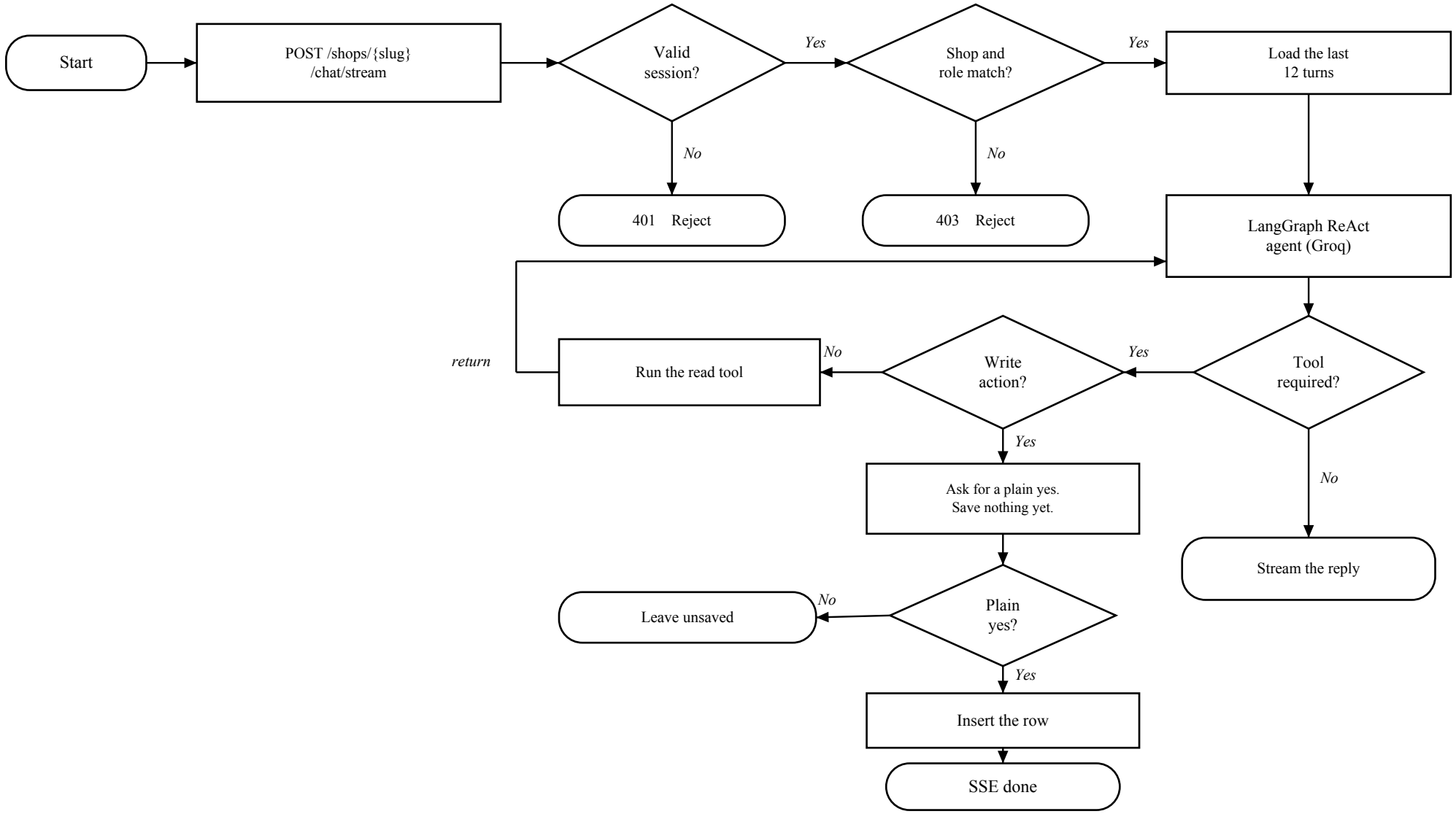


Figure 1. Customer chat, from the HTTP request to a saved row. Admin chat does not use the confirm step. See section 6.

6. Admin chat

Admin chat is a second agent, at *POST /shops/{slug}/admin/chat/stream*. It does not save messages. Every tool runs a SELECT only. The reply has to name the time window it used.

Shop	Tools
Himali Tea House	order_stats, top_products, client_stats, conversion
Lotus Wellness Studio	booking_stats, top_services, client_stats, conversion

The period is one of *today*, *this_week*, *last_7_days*, or *all*. Studio times use Asia/Kathmandu. A tea admin who calls the studio admin route receives 403.

7. Retrieval

One knowledge row is one chunk, and one chunk is one section of a document. There is no fixed character size and no overlap between chunks.

The string that is embedded is the document name and the section name, repeated five times, followed by the section body. Repeating the heading makes the title count for more than the paragraph when two vectors are compared.

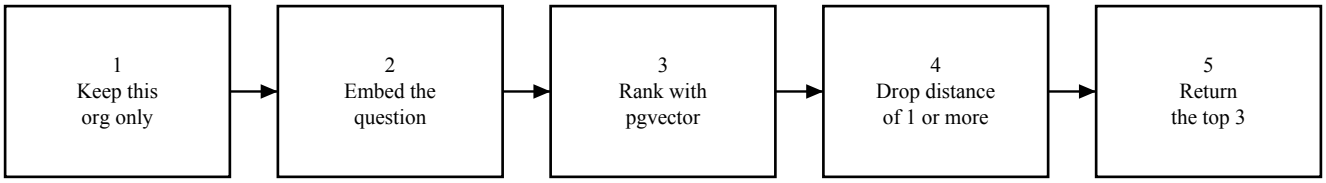


Figure 2. Search order. Rows are limited to the session organisation before they are ranked.

On the live site the question is embedded with Gemini *gemini-embedding-001* (1,024 dimensions) when *GEMINI_API_KEY* is set. Tests, and any run with no key, use a local hash embedder of the same width. Distance is the pgvector operator <=>. A row at distance 1 or more is dropped. The limit is 3. The reply may cite the document name and the section. An organisation id passed inside the tool arguments is ignored.

8. HTTP API

Customer routes

Method	Path	What it does
POST	/login	Start a session. Returns token, orgId, and role.
POST	/logout	Delete the session row.
POST	/shops/{slug}/signup	Create a customer in that shop.
POST	/shops/{slug}/chat	One JSON reply, no stream.
POST	/shops/{slug}/chat/stream	Customer agent. Server-sent events.
GET	/me/orders	This customer's tea orders.
GET	/me/bookings	This customer's studio bookings.
GET	/conversations	List this user's conversations.
POST	/conversations	Open a conversation.
PATCH	/conversations/{id}	Rename a conversation.
DELETE	/conversations/{id}	Delete a conversation.

Admin routes

Method	Path	What it does
POST	/shops/{slug}/admin/chat/stream	Read-only stats agent.
GET	/admin/orders	Tea orders for this org. Admin only.
GET	/admin/bookings	Studio bookings for this org. Admin only.
POST	/shops/himali-tea/orders/{id}/status	Set status, for example shipped.

The slug is *himali-tea* or *lotus-studio*.

9. Requests worth trying

Use the live site or the local API. Send the bearer token on every call after login. Prices are in NPR.

No.	Call	What you should see
1	POST /login as bishal@tea.com	200, and orgId org_a.
2	Tea stream: "price of jasmine"	token events, then done.
3	Same tea token on lotus-studio chat	403.
4	Tea stream: order one Jasmine Green 50g, then yes on the same conversationId	GET /me/orders shows a new pending order.
5	POST /login as admin@tea.com	200, role admin, org_a.
6	Tea admin stream: "How many orders today?"	A reply that names the time window.
7	That admin token on the studio admin stream	403.

Login, then a price question.

```
POST /login
{ "email": "bishal@tea.com", "password": "devpass" }

POST /shops/himali-tea/chat/stream
Authorization: Bearer <token>
{ "message": "price of jasmine" }
```

An order is two turns. Take *conversationId* from the meta event of the first turn and send it back with yes.

```
{ "message": "Please order one Jasmine Green 50g" }
{ "message": "yes", "conversationId": "<id from meta>" }
GET /me/orders
```

Studio checks use bishal@studio.com and the slug lotus-studio. "How many bookings this week?" is for admin@studio.com.

10. Evaluation

The command *make eval* runs *python -m eval.run_eval*. There are 12 tea cases and 12 studio cases. The runner uses a fake tool planner. It does not call Groq. After each case it checks the reply text and the rows in Postgres.

The set covers citations, questions the notes do not answer, the confirm step before a save, stock limits, cancel rules, reorder, booking conflicts, opening hours, a call into the other shop, and prompt injection.

Unit tests are separate. From the project directory, with Postgres listening on port 5433:

```
DATABASE_URL=postgresql://shop:shop@localhost:5433/shop \
PYTHONPATH=.pydeps python3 -m pytest -q
```